



Full-Stack Development with Java, React & MongoDB

DAY 1

Day 1 – What We're Covering Today

01

How the Web Works

Understanding browsers, servers, and requests

02

HTTP & APIs

The language that powers the internet

03

JSON

The data format used by modern APIs

04

Git & Version Control

Tracking changes like a professional

05

Hands-On Labs

Create a repo, call an API, write JSON

COURSE OVERVIEW

Full-Stack Development with Java, React & MongoDB

Duration

20 Days of guided, hands-on learning

Focus Areas

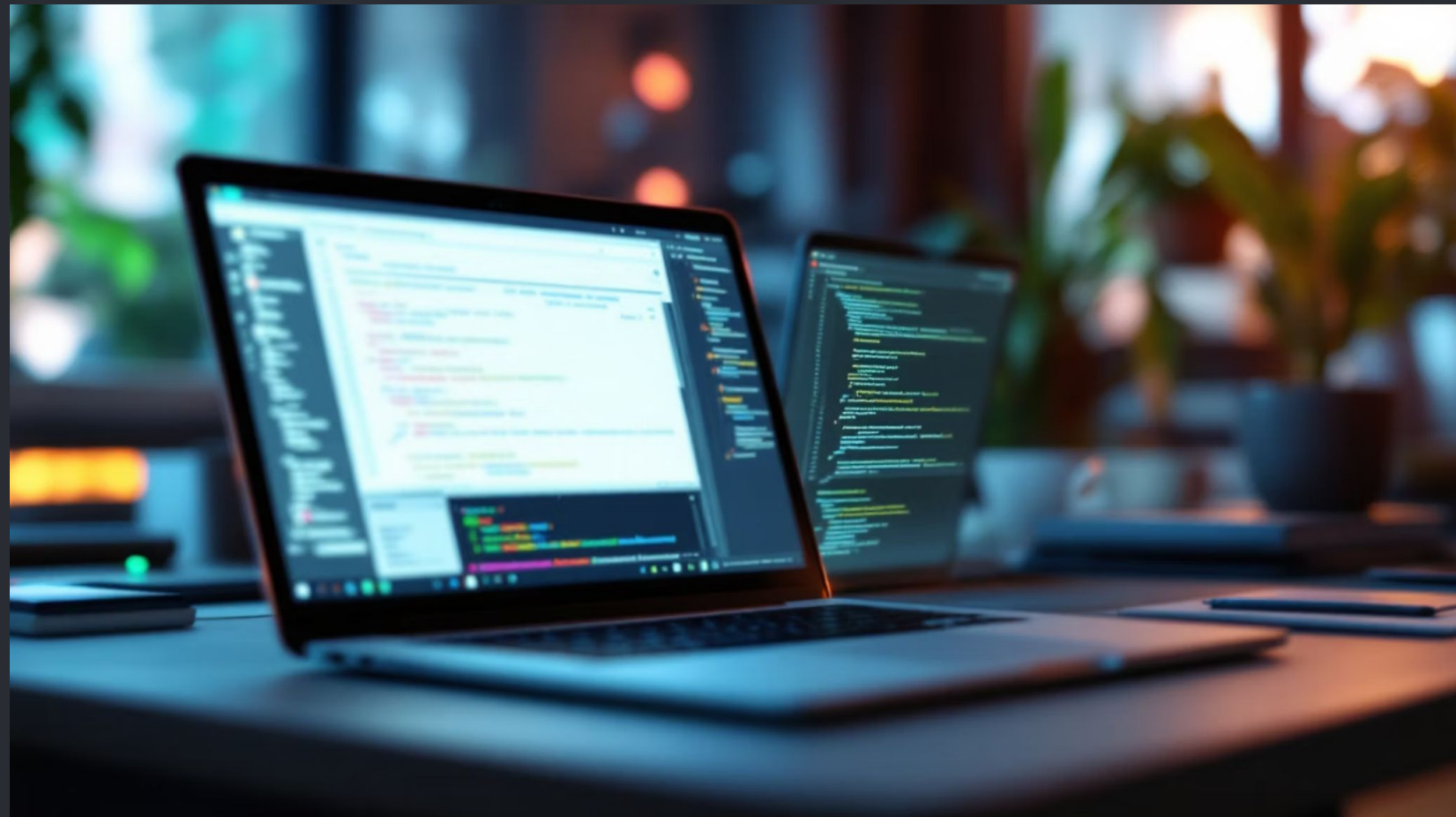
Web fundamentals, dev tools, APIs, Git

Final Output

A working capstone application

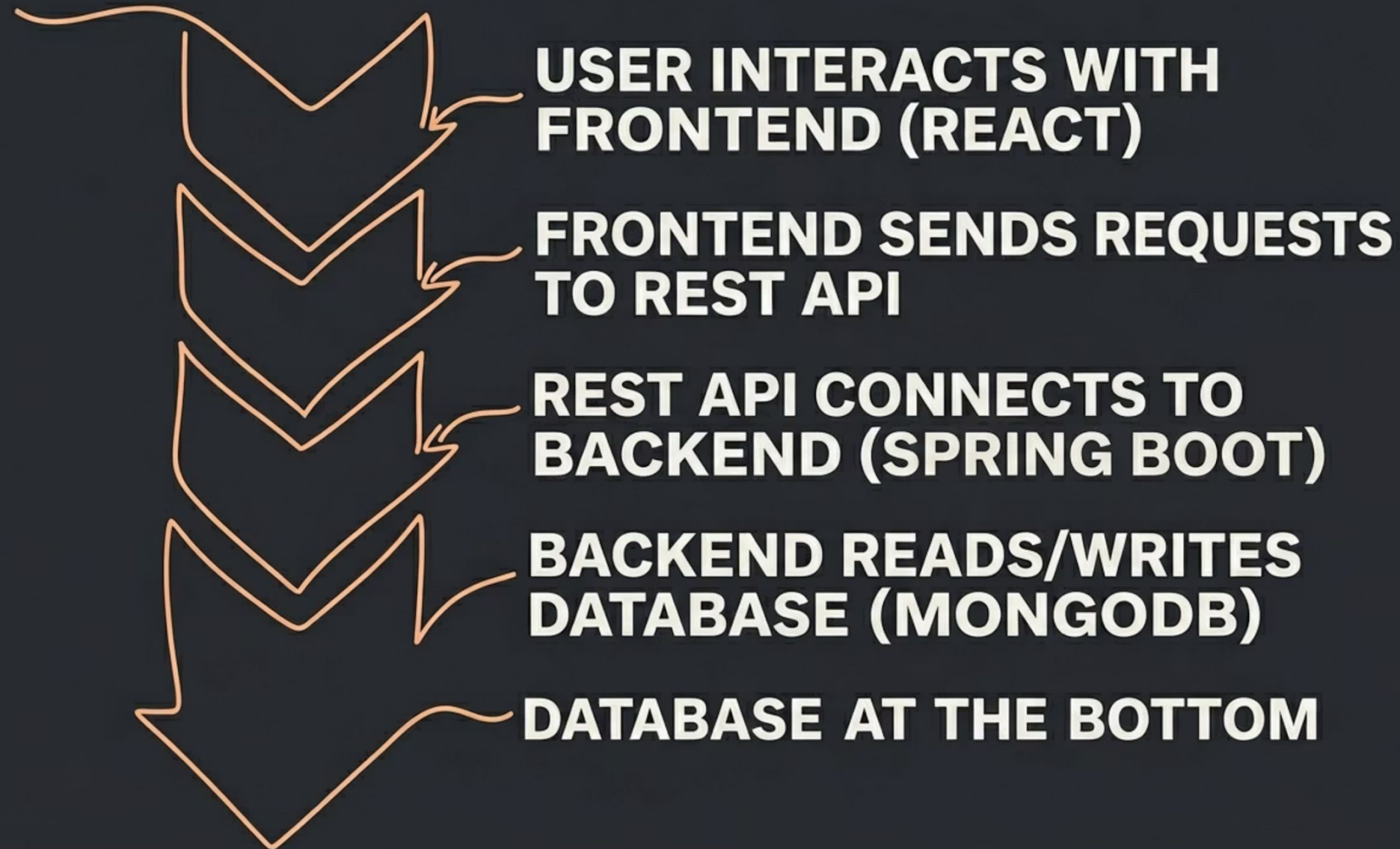


By the End of This Programme, You Will Be Able To...



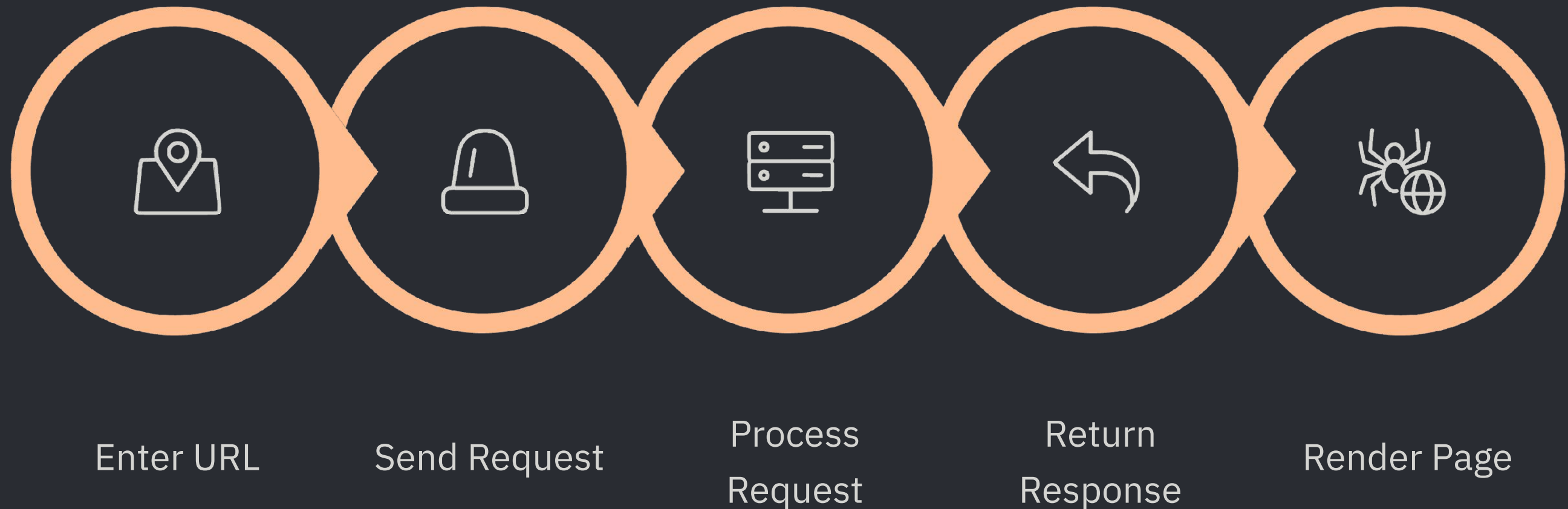
- Build a full-stack web application
- Develop REST APIs using Spring Boot
- Create React user interfaces
- Store and query data in MongoDB
- Deploy and test your applications

Full-Stack Architecture — How It All Fits Together



Each layer has a clear responsibility — the **frontend** presents data, the **API** routes requests, the **backend** processes logic, and the **database** persists information.

What Happens When You Open a Website?



Every page load triggers this cycle. The example request `GET https://api.github.com/users` follows exactly these five steps — understanding this is the foundation of web development.

Client vs. Server

Client

The part the user sees and interacts with directly.

- Browser (Chrome, Firefox)
- Mobile app
- React frontend

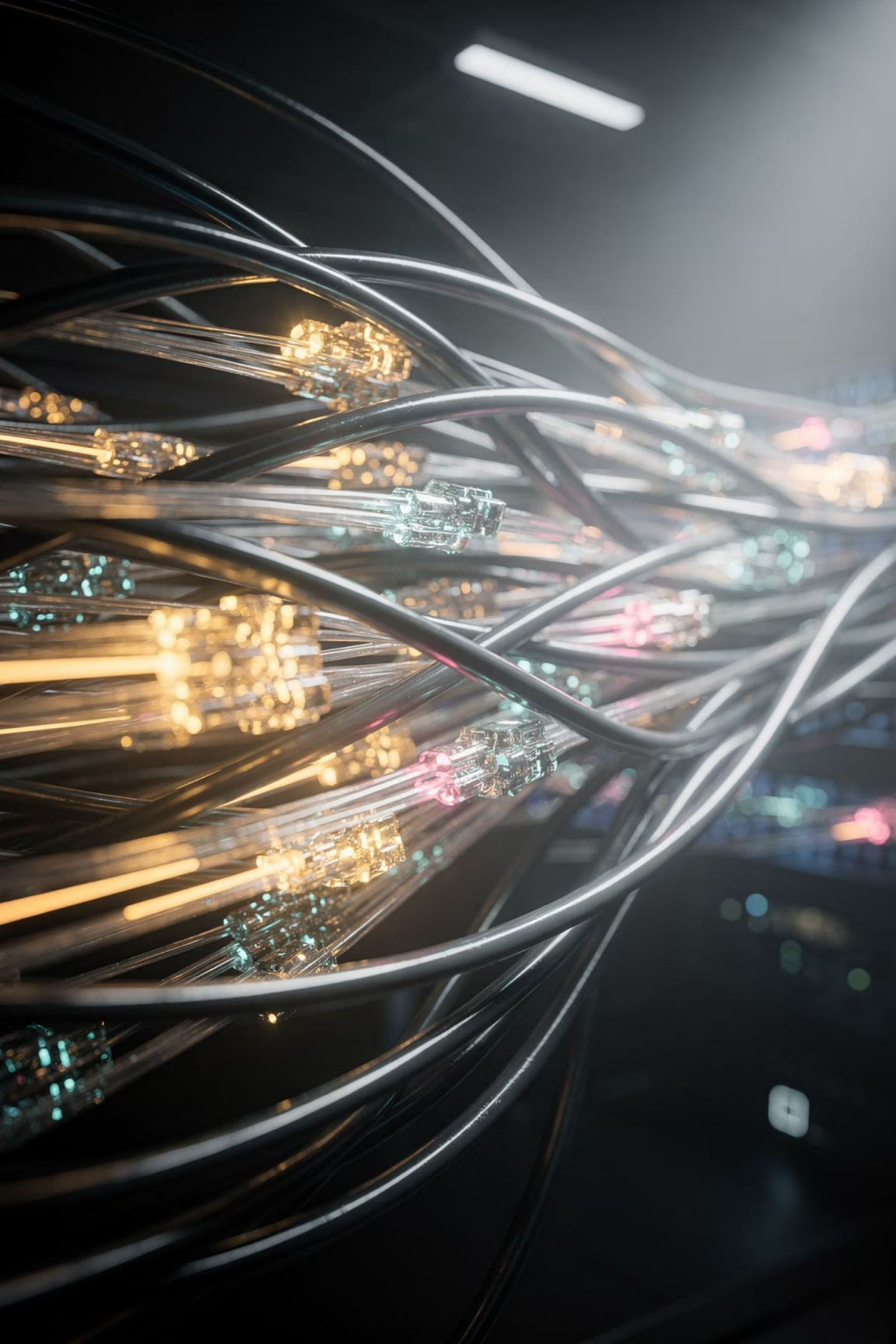
Server

The hidden engine that powers the application.

- Backend application (Spring Boot)
- Handles business logic
- Reads and writes to the database



Flow: **Browser** → **API** → **Database**



HTTP

HTTP Basics — The Language of the Web

HTTP (HyperText Transfer Protocol) is how browsers and servers communicate. Every interaction on the web is an HTTP conversation.

GET

Retrieve data

POST

Create data

PUT

Update data

DELETE

Remove data

HTTP Request Structure

Example Request

```
GET /users HTTP/1.1  
Host: api.example.com  
Accept: application/json
```

The Four Parts

Method

What action to perform
(GET, POST...)

Endpoint

The resource path (e.g.
`/users`)

Headers

Metadata about the
request

Body

Optional — used with
POST/PUT

HTTP Response Structure

Example Response

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "name": "Alice",
  "age": 25
}
```

Three Key Parts

- 1 Status Code**
`200 OK` — tells you if the request succeeded or failed
- 2 Headers**
Metadata such as content type and encoding
- 3 Response Body**
The actual data returned — usually JSON

Common HTTP Status Codes

200

OK — Request succeeded

201

Created — New resource made

400

Bad Request — Something's wrong with your request

401

Unauthorised — Login required

404

Not Found — Resource doesn't exist

500

Server Error — Something broke on the server

Status codes are your first clue when debugging — always check the code before reading the body.

JSON

Introduction to JSON

JSON (JavaScript Object Notation) is the most widely used data format for APIs. It's human-readable and lightweight.

```
{  
  "name": "John",  
  "age": 30,  
  "skills": ["Java", "React"]  
}
```

```
{  
  "apadrook": {  
    oppes: "object" {  
      "apge::mok";  
      "dijectrook" {  
        "objects: { "andends"  
          {  
            "laad: "obyecdt";  
            "likgobook";  
            "modtr-:object";  
            "robject::"obyecdt" {  
              "rlegs::monst";  
            }  
          }  
        }  
      }  
    }  
  }  
}
```

JSON Building Blocks

Three Core Concepts

Objects { }

A collection of key-value pairs wrapped in curly braces

Arrays []

An ordered list of values wrapped in square brackets

Key-Value Pairs

`"name": "John"` — a named field and its value

Full Example

```
{  
  "name": "John",  
  "age": 30,  
  "active": true,  
  "skills": [  
    "Java",  
    "React"  
  ]  
}
```

Why JSON Is the Industry Standard



Easy to Read

Developers can understand it at a glance — no special tools needed



Lightweight

Minimal syntax overhead means faster data transfer



Universal

Supported natively by every modern programming language



API-Ready

The default format for REST APIs worldwide

JSON vs XML

Before JSON, developers used **XML** — verbose and harder to read.

```
<!-- XML (verbose) -->
```

```
<user>
```

```
  <name>John</name>
```

```
</user>
```

```
// JSON (clean)
```

```
{ "name": "John" }
```

GIT

Introduction to Git — Version Control for Developers

Git is a **version control system** that tracks every change you make to your code. It's used by virtually every software team in the world.

Track Changes

See exactly what changed, when, and who made the change

Collaborate

Multiple developers can work on the same project simultaneously

Restore History

Roll back to any previous version if something goes wrong





The Basic Git Workflow



Think of a **commit** as a save point in a game — you can always return to it. Each commit captures the full state of your project at that moment.

Essential Git Commands

```
git init  
git add .  
git commit -m "message"  
git branch feature-name  
git checkout main  
git push origin main
```

`git init`

Initialises a new local repository

`git add`

Stages files ready to be committed

`git commit`

Saves a snapshot of staged changes

`git branch`

Creates an isolated line of development

`git push`

Uploads commits to a remote (e.g. GitHub)

Why Every Developer Uses Git



Collaboration

Teams work on the same codebase
without overwriting each other's work



Full History

Every change is logged — see what
changed, when, and why



Safe Experimentation

Use branches to try new features without
risking the main codebase

Calling a Public API

Try It Now

Open your browser and visit:

 <https://jsonplaceholder.typicode.com/posts>

This is a free, public REST API that returns sample data — perfect for practising JSON and HTTP requests without setting up a backend.

What You'll See

```
[
  {
    "id": 1,
    "title": "My First Post",
    "body": "This is a post",
    "userId": 1
  },
  ...
]
```

The API returns an **array of JSON objects**. Each object represents one post.

Example JSON Response — Decoded

Raw Response

```
{  
  "id": 1,  
  "title": "My First Post",  
  "body": "This is a post",  
  "userId": 1  
}
```

How a Frontend Uses This

- Fetch the JSON from the API
- Parse each field (id, title, body)
- Render the data in the UI (e.g. a blog page)

This is exactly how React apps consume backend data in the real world.

LAB TIME

Today's Lab Overview

1

Create a Git Repository

Initialise your first local repo from the terminal

2

Make Your First Commit

Stage and commit a README file

3

Call a Public API

Fetch live data from JSONPlaceholder

4

Analyse the JSON

Identify fields, arrays, and object structure



MINI QUIZ

Check Your Understanding

1

What does HTTP stand for?

Hint: it's the protocol that powers every web request

2

What does a GET request do?

Hint: think about reading vs. writing

3

What format do APIs usually return?

Hint: you learned about it today — lightweight and readable

4

What command initialises a Git repository?

Hint: it's the very first Git command you'll ever run

DAY 1 WRAP-UP

Key Takeaways

How the Web Works

Requests, responses, clients, and servers

HTTP Basics

Methods, status codes, request & response structure

JSON

The universal data format for modern APIs

Git

Version control, commits, and the developer workflow

 **Tomorrow:** Java Programming Fundamentals — variables, methods, and object-oriented concepts.





LAB WORKSHEET — DAY 1

Hands-On Lab Exercises

Work through each lab in order. Ask your instructor if you get stuck. The goal is to get comfortable with the terminal, Git, and reading JSON.

Create Your First Git Repository

Step-by-Step

- 1 Open your terminal
- 2 Create and enter your project folder
- 3 Initialise Git
- 4 Create a README and commit

Commands

```
mkdir fullstack-course  
cd fullstack-course  
git init  
echo "Fullstack Training Repo" > README.md  
git add .  
git commit -m "First commit"
```



Expected result: Repository created with one commit in history.

Explore a Public API

Instructions

Open your browser or Postman and visit:

 <https://jsonplaceholder.typicode.com/posts>

01

Count the objects returned

02

Identify the fields in each object

03

Sketch the JSON structure on paper

Example Answer

```
{
  "id": 1,
  "title": "...",
  "body": "...",
  "userId": 1
}
```

You should see **100 posts** returned. Each object has 4 fields: `id`, `title`, `body`, `userId`.

Convert Data to JSON

Raw Data

Name

Sarah

Age

22

Course

Computer Science

Skills

Java, Python, JavaScript

Expected JSON Output

```
{
  "name": "...",
  "age": num,
  "course": "their_cpirse",
  "skills": [
    "skill1",
    "skill2",
    "skill3"
  ]
}
```

Notice that `skills` is an **array** because there are multiple values.

LAB 4

Inspect an API Response with DevTools

Steps

1 Open DevTools

Press **F12** or right-click → Inspect

2 Go to the Network tab

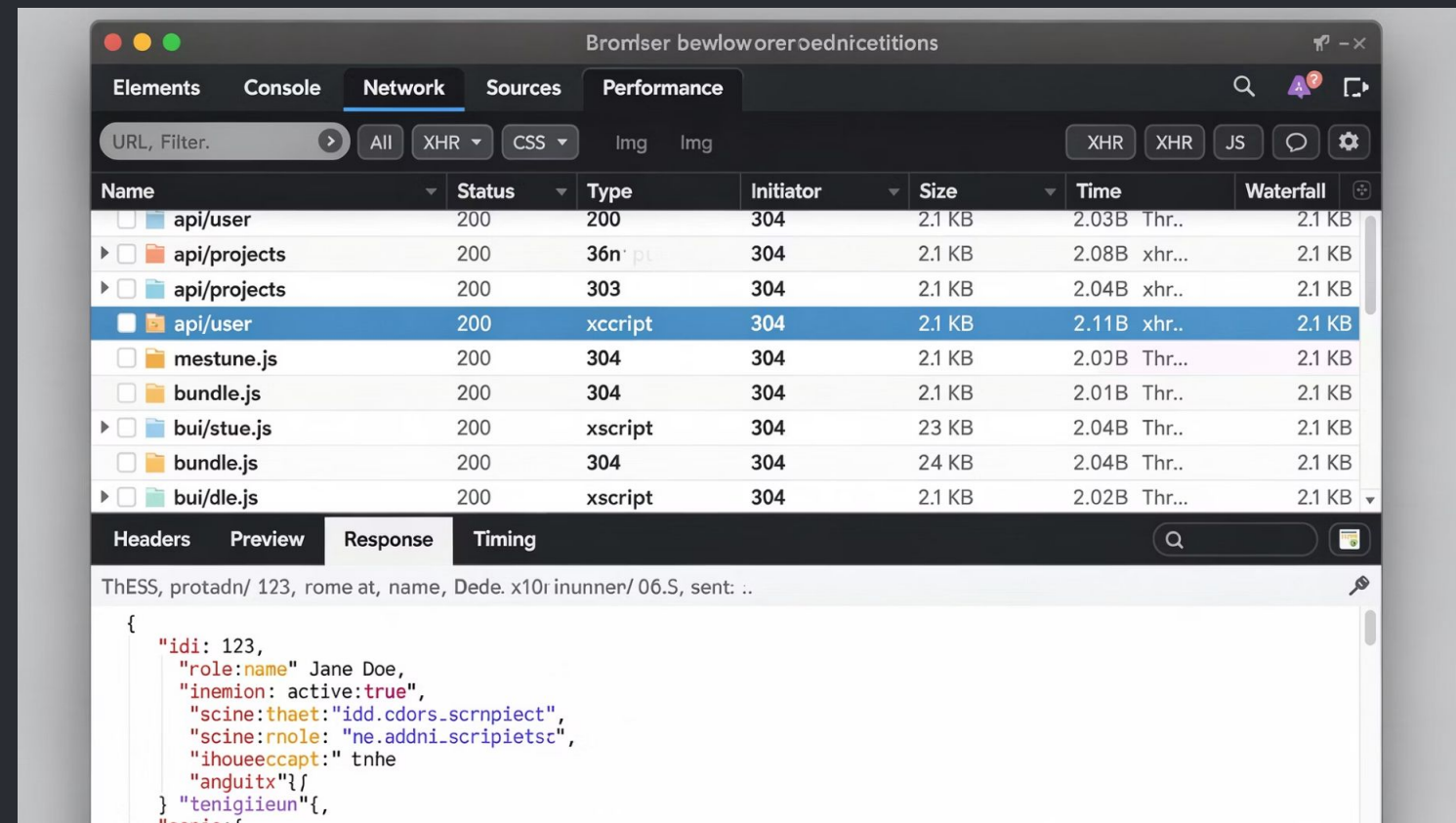
This records all HTTP activity

3 Reload the page

Watch requests appear in real time

4 Click a request and inspect it

Find the method, status code, and response body



END OF DAY 1

End-of-Day Reflection

Take a few minutes to write down your answers before leaving today's session.

1

What happens when you open a website?

Describe the journey from URL to displayed page in your own words.

2

Why is JSON useful?

What makes it better suited for APIs than alternatives like XML?

3

Why do developers use Git?

Name at least two benefits Git brings to a development team.

